

23CS201 - Data Structures and Algorithms using C++

Study Materials

Basics of C++:

- C++ is a versatile programming language known for its efficiency, flexibility,
- and wide applicability. It combines procedural, object-oriented, and generic
- programming features, allowing developers to create high-performance
- software across various domains. With its rich standard library and support
- for low-level memory manipulation, C++ is commonly used in system
- software, game development, embedded systems, and more. Its syntax is
- based on C, enhancing it with features like classes, inheritance,
- polymorphism, templates, and exception handling, making it a powerful
- tool for software development.

Arrays:

- The array stores a fixed-size sequential collection of elements of the same type.
- An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.
- Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ...numbers[99] to represent individual variables.
- A specific element in an array is accessed by an index.
- All arrays consist of contiguous memory locations.
- The lowest address corresponds to the first element and the highest address to the last element.

Types of array:

- There are two types of arrays:
- One Dimensional Array
- Multi Dimensional Array
- **One Dimensional Array:** A one dimensional array is a collection of same data types.
- 1-D array is declared as:

```
data_type variable_name[size]
```

`data_type` is the type of array, like int, float, char, etc.

`variable_name` is the name of the array.

`size` is the length of the array which is fixed.

Representation of Array:

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

Multi-Dimensional Array:

- A multidimensional array is also known as array of arrays.
- Generally, we use a two-dimensional array.
- It is also known as the matrix.
- We use two indices to traverse the rows and columns of the 2D array.
- It is declared as:

```
data_type variable_name[N][M]
```

`data_type` is the type of array, like `int`, `float`, `char`, etc.

`variable_name` is the name of the array.

`N` is the number of rows.

`M` is the number of columns.

Multi-Dimensional Array:

	Col1	Col2	Col3	Col4
Row1	Arr[0][0]	Arr[0][1]	Arr[0][2]	Arr[0][3]
Row2	Arr[1][0]	Arr[1][1]	Arr[1][2]	Arr[1][3]
Row3	Arr[2][0]	Arr[2][1]	Arr[2][2]	Arr[2][3]
Row4	Arr[3][0]	Arr[3][1]	Arr[3][2]	Arr[3][3]

Strings:

- C++ strings are sequences of characters stored in a char array.
- Strings are used to store words and text.
- They are also used to store data, such as numbers and other types of information.
- Strings in C++ can be defined either using the `std::string` class or the C-style character arrays.
- This class provides a convenient way to manipulate strings, including operations like concatenation, substring extraction, and comparison.
- Strings are fundamental for representing and processing textual data in computer programs.

Declaration and Initialization::

```
std::string str1;          // Declares an empty string
```

```
std::string str2 = "Hello"; // Initializes string with "Hello"
```

```
std::string str3(5, 'A');   // Initializes string with 5 'A's
```

Input and Output:

```
std::string str;
```

```
std::cin >> str;          // Input a string
```

```
std::cout << str;         // Output a string
```

String Concatenation:

String Concatenate

"Hello" + "World" = "Hello World"



String 1 String 2 Result

String Manipulation:

- `substr()`: Extracts a substring from a string.
- `find()`: Finds the first occurrence of a substring within a string.
- `replace()`: Replaces a portion of the string with another string.
- `erase()`: Removes characters from the string.
- `insert()`: Inserts characters into the string.
- Conversion:
- `c_str()`: Converts a `std::string` to a C-style string (`const char*`).
- `std::to_string()`: Converts numeric types to string

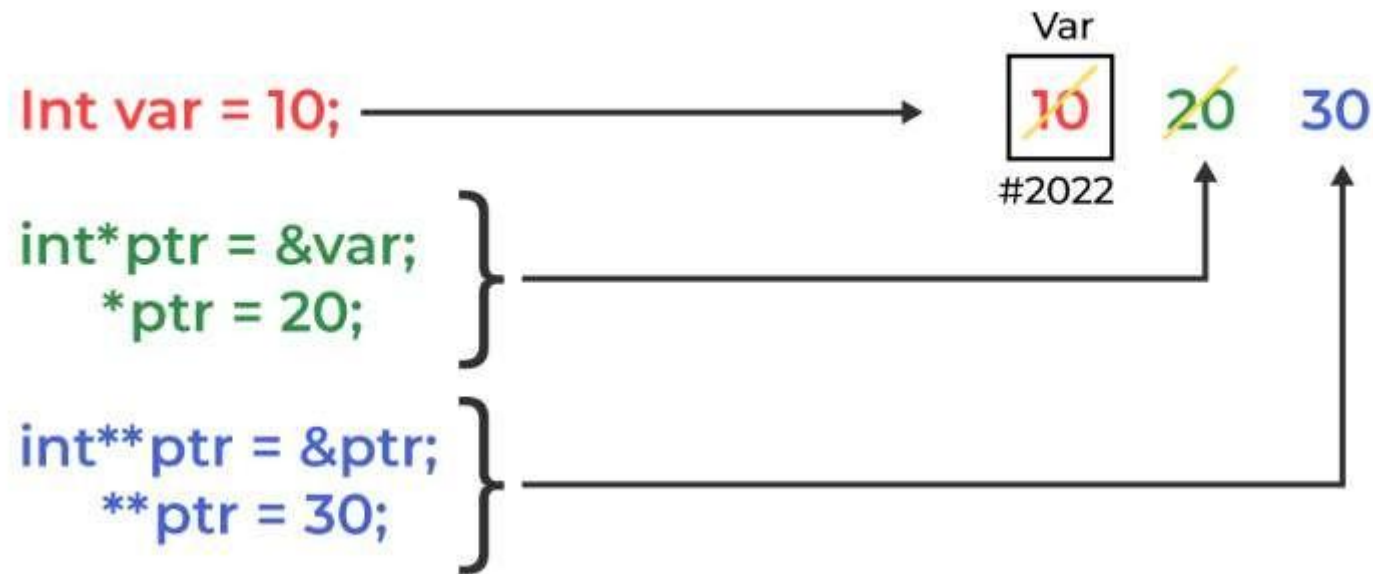
Pointers:

- Pointers are symbolic representations of addresses.They enable programs to simulate call-by-reference as well as to create and manipulate dynamic data structures.
- Iterating over elements in arrays or other data structures is one of the main use of pointers.
- The address of the variable you're working with is assigned to the pointer variable that points to the same data type (such as an int or string).
- **Syntax:**

```
datatype *var_name;  
int *ptr;    // ptr can point to an address which holds int data
```

Steps to Solve RMQ Using a Segment Tree:

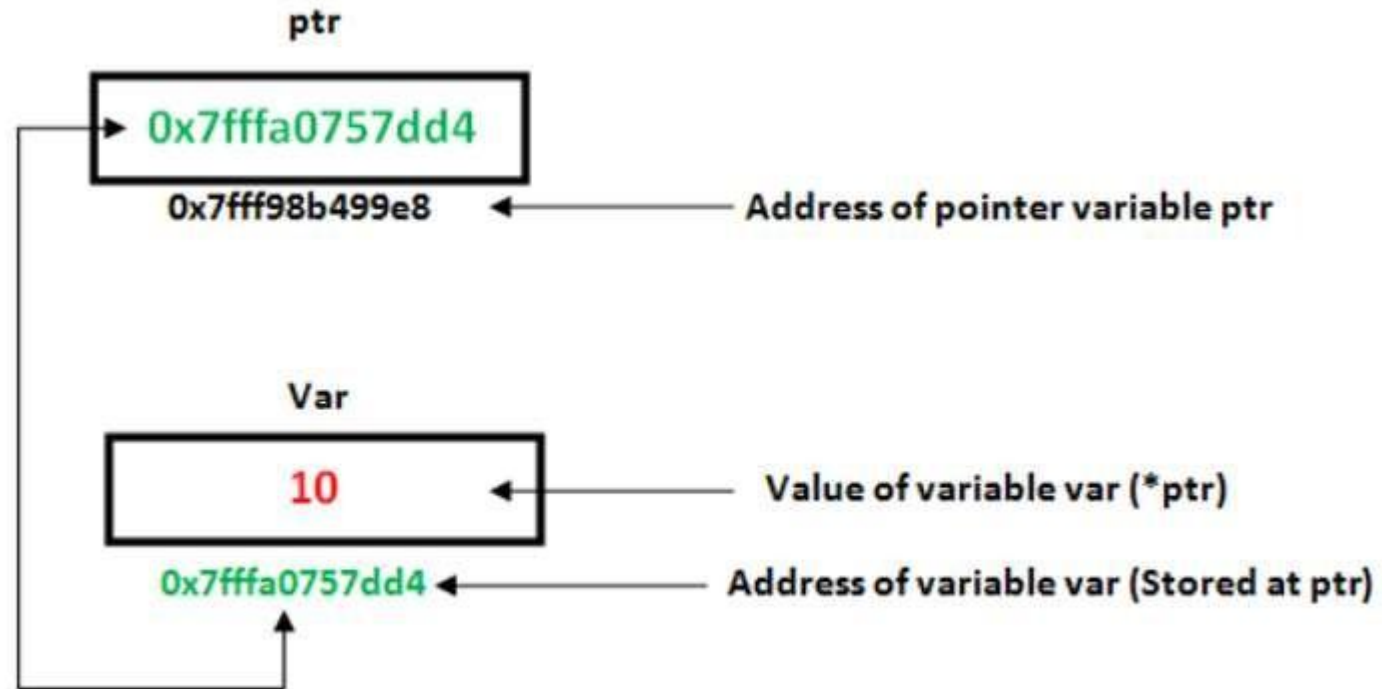
How Pointer Works in C++



How to use a pointer?:

- Define a pointer variable
- Assigning the address of a variable to a pointer using the unary operator (&) which returns the address of that variable.
- Accessing the value stored in the address using unary operator (*) which returns the value of the variable located at the address specified by its operand.
- The reason we associate data type with a pointer is that it knows how many bytes the data is stored in. When we increment a pointer, we increase the pointer by the size of the data type to which it points.

Pointers:



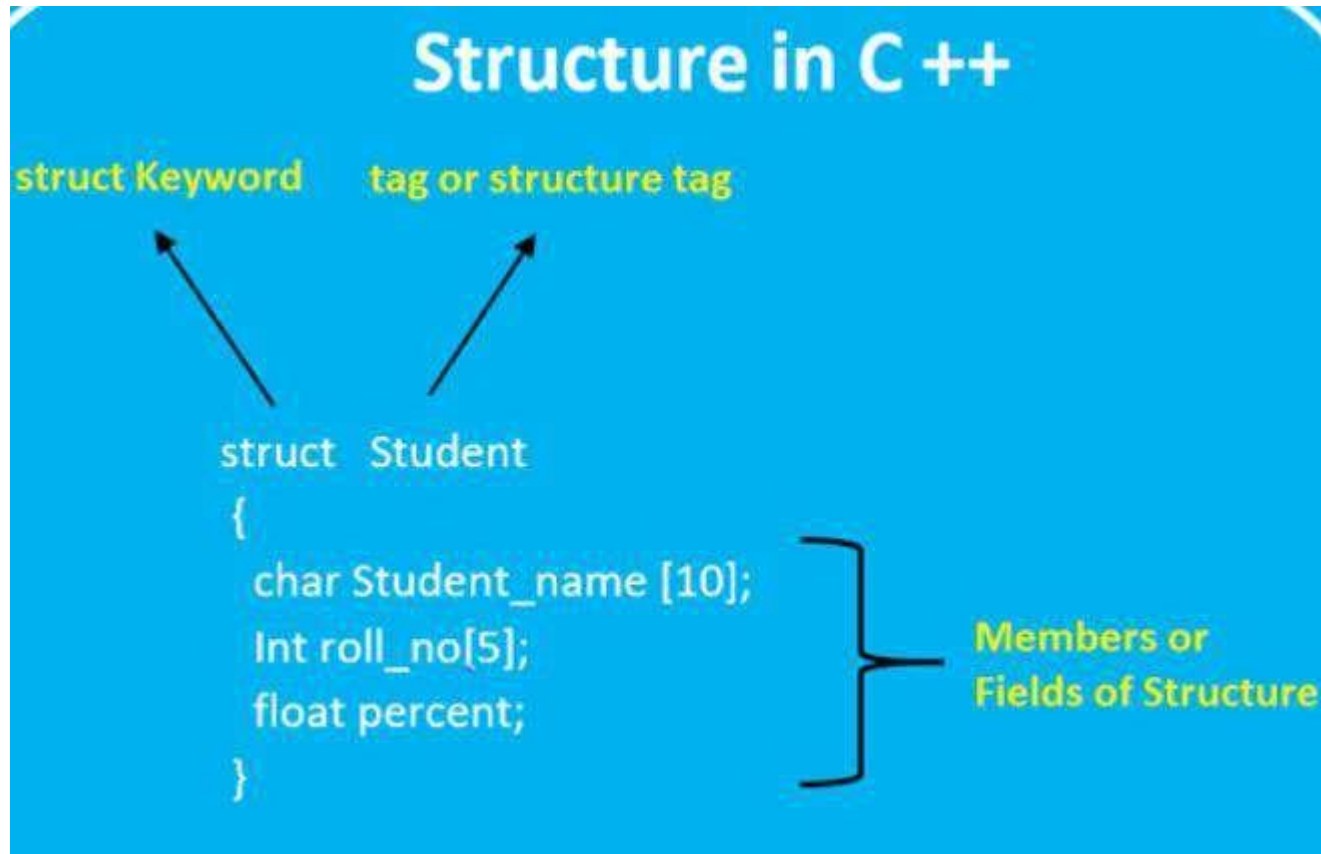
References and Pointers:

- There are 3 ways to pass C++ arguments to a function:
- Call-By-Value
- Call-By-Reference with a Pointer Argument
- Call-By-Reference with a Reference Argument

Structures:

- A structure is a user-defined data type in C/C++.
- A structure creates a data type that can be used to group items of possibly different types into a single type.
- The C++ Course covers how to define and use structures effectively in C++, helping you manage complex data efficiently.
- A structure is a user-defined data type that groups related variables of different types.
- Key Points: Structures are used to represent records.

Structures:



Declaration:

```
struct tag
{
    data_type1 member1;
    data_type2 member2;
    data_type3 member3;
};
```

```
struct <structure_name>
{
    data_type1 member1;
    data_type2 member2;
    data_type3 member3;
    ...
    ...
};
```

Difference Between Array and Structure:

Array	Structure
Array is collection of homogeneous data.	Structure is the collection of heterogeneous data.
Array data are access using index.	Structure elements are access using . (dot) operator.
Array allocates static memory.	Structures allocate dynamic memory.
To access Array element need less time compare to structures.	To access Structure elements takes more time compare to Array.